Vigenère, RSA, CBC/CFB, ElGamal

1. Vigenère – Key Recovery (Known Length)

Given Data:

- **Ciphertext (C):** `LXFOPVEFRNHR`
- **Key Length (L):** 5
- **Context:** English language, no punctuation.

a) Find the keys using Columnar Analysis (Caesar Type)

Since the key length is 5, the Vigenère cipher behaves like 5 interleaved Caesar ciphers. We arrange the ciphertext into 5 columns and analyze each column to find a shift (Key letter) that produces high-frequency English letters (e.g., A, E, T, N, O).

1. Splitting Ciphertext into Columns:

Col 1 (K_1)	**Col 2 (K_2)**	**Col 3 (K_3)**	**Col 4 (K_4)**	**Col 5 (K_5)**
L	X	F	O	P
V	E	F	R	N
H	R	R		

2. Analysis per Column:

- **Column 1 (L, V, H):** Testing Key letter **L** (11).

$$L - L = A, \quad V - L = K, \quad H - L = W$$

  Result: **A, K, W**. (Produces 'A').

- **Column 2 (X, E, R):** Testing Key letter **E** (4).

$$X - E = T, \quad E - E = A, \quad R - E = N$$

  Result: **T, A, N**. (Excellent frequency distribution).

- **Column 3 (F, F, R):** Testing Key letter **M** (12).

$$F - M \equiv T, \quad R - M = F$$

  Result: **T, T, F**. ('T' is very common).

- **Column 4 (O, R):** Testing Key letter **O** (14).

$$O - O = A, \quad R - O = D$$

  Result: **A, D**. ('A' is very common).

- **Column 5 (P, N):** Testing Key letter **N** (13).

$$P - N = C, \quad N - N = A$$

  Result: **C, A**. ('A' is very common).

Recovered Key: `LEMON`

b) Decrypt the message

Using the recovered key **LEMON**, we perform the decryption operation:

$$P_i = (C_i - K_i) \pmod{26}$$

Index	**Cipher (C)**	**Key (K)**	**Numeric Calc**	**Result**	**Plaintext (P)**
1	L (11)	L (11)	$11 - 11 = 0$	0	**A**
2	X (23)	E (4)	$23 - 4 = 19$	19	**T**
3	F (5)	M (12)	$5 - 12 = -7 \equiv 19$	19	**T**
4	O (14)	O (14)	$14 - 14 = 0$	0	**A**
5	P (15)	N (13)	$15 - 13 = 2$	2	**C**
6	V (21)	L (11)	$21 - 11 = 10$	10	**K**
7	E (4)	E (4)	$4 - 4 = 0$	0	**A**
8	F (5)	M (12)	$5 - 12 = -7 \equiv 19$	19	**T**
9	R (17)	O (14)	$17 - 14 = 3$	3	**D**
10	N (13)	N (13)	$13 - 13 = 0$	0	**A**
11	H (7)	L (11)	$7 - 11 = -4 \equiv 22$	22	**W**
12	R (17)	E (4)	$17 - 4 = 13$	13	**N**

Final Decrypted Message:

`ATTACK AT DAWN`

2. RSA Full Cycle with Fast Exponentiation

Given Data:

- Primes: $p = 197$, $q = 211$
- Public Exponent: $e = 24377$
- Message: $m = 1234$

a) Calculate n, $\phi(n)$, and d

1. **Modulus n:**

$$n = p \cdot q = 197 \cdot 211 = \mathbf{41567}$$

2. **Euler's Totient Function $\phi(n)$:**

$$\phi(n) = (p-1)(q-1) = 196 \cdot 210 = \mathbf{41160}$$

3. **Private Exponent d:** We must solve $e \cdot d \equiv 1 \pmod{\phi(n)}$, i.e., $24377 \cdot d \equiv 1 \pmod{41160}$. Using the Extended Euclidean Algorithm:

$$41160 = 1 \cdot 24377 + 16783$$

$$...(IntermediateEuclideansteps)...$$

$$1 = 17393 \cdot 24377 - 10301 \cdot 41160$$

The coefficient of e is the inverse:

$$\mathbf{d = 17393}$$

b) Encryption and Decryption

Encryption (C):

$$C = m^e \pmod{n} \Rightarrow C = 1234^{24377} \pmod{41567} = \mathbf{34503}$$

Decryption (M):

$$M = C^d \pmod{n} \Rightarrow M = 34503^{17393} \pmod{41567} = \mathbf{1234}$$

Verification successful: The decrypted message matches the original.

c) Square-and-Multiply Steps

We demonstrate the steps for the exponent $e = 24377$.
Binary representation: 101111100010001_2.

Bit	Operation	Calculation (mod 41567)
1	Start	1234
0	Square	$1234^2 = 18469$
1	Sq & Mul	$18469^2 \cdot 1234 = 29547$
1	Sq & Mul	$29547^2 \cdot 1234 = 27488$
1	Sq & Mul	$27488^2 \cdot 1234 = 17498$
1	Sq & Mul	$17498^2 \cdot 1234 = 10419$
1	Sq & Mul	$10419^2 \cdot 1234 = 34749$
0	Square	$34749^2 = 33061$
0	Square	$33061^2 = 19293$
0	Square	$19293^2 = 22904$
1	Sq & Mul	$22904^2 \cdot 1234 = 22718$
0	Square	$22718^2 = 2070$
0	Square	$2070^2 = 40375$
0	Square	$40375^2 = 36940$
1	Sq & Mul	$36940^2 \cdot 1234 = \mathbf{34503}$

3. Block Cipher Modes CBC & CFB – Error Propagation

Scenario: A 128-bit Block Cipher is used. A transmission error corrupts **exactly one bit** in ciphertext block C_1 (denoted as C_1'). We analyze the effect on decrypted blocks P_1' and P_2'.

A. CBC Mode (Cipher Block Chaining)

Decryption Equation: $P_i = D_k(C_i) \oplus C_{i-1}$.

1. **Effect on P_1':**
$$P_1' = D_k(C_1') \oplus IV$$
The corrupted C_1' enters the decryption function D_k. Due to the **Avalanche Effect** inherent in block ciphers, a 1-bit change in input results in approximately 50% of the output bits flipping.
Result: The entire block P_1' is **completely corrupted** (random garbage).

2. **Effect on P_2':**
$$P_2' = D_k(C_2) \oplus C_1'$$
The term $D_k(C_2)$ is correct. It is XORed with C_1'. Since the XOR operation is bitwise linear, the single bit error in C_1' is transferred directly to the result.
Result: P_2' contains exactly **1 bit error** (at the exact same position as the error in C_1').

B. CFB Mode (Cipher Feedback)

Decryption Equation: $P_i = C_i \oplus E_k(C_{i-1})$.

1. **Effect on P_1':**
$$P_1' = C_1' \oplus E_k(IV)$$
The keystream $E_k(IV)$ remains correct. It is XORed with C_1'. The single bit error propagates directly.
Result: P_1' contains exactly **1 bit error** (at the exact same position as the error in C_1').

2. **Effect on P_2':**
$$P_2' = C_2 \oplus E_k(C_1')$$
The corrupted C_1' enters the encryption function E_k to generate the next keystream. Due to the **Avalanche Effect**, the output $E_k(C_1')$ is completely different from the expected keystream.
Result: The entire block P_2' is **completely corrupted** (random garbage).

4. ElGamal Numerical Example

Given Data:

- Prime $p = 23$, Generator $g = 5$
- Private Key $x = 6$
- Ciphertext $C = (C_1, C_2) = (20, 22)$

a) Calculate $C_1^x \pmod p$ (Shared Secret)

We calculate $S = C_1^x \pmod p$:

$$S = 20^6 \pmod{23}$$

Using the property $20 \equiv -3 \pmod{23}$:

$$S \equiv (-3)^6 \equiv 729 \pmod{23}$$

Performing the modulo operation ($729 = 31 \cdot 23 + 16$):

$$\mathbf{S = 16}$$

b) Find Message M

The decryption formula is $M = C_2 \cdot (S)^{-1} \pmod p$. First, we need the modular inverse of $16 \pmod{23}$. From section (c), we know $16^{-1} \equiv 13 \pmod{23}$.

Substitute back into the equation:

$$M = 22 \cdot 13 \pmod{23}$$

Using $22 \equiv -1 \pmod{23}$:

$$M \equiv -1 \cdot 13 \equiv -13 \pmod{23}$$

Converting to positive residue:

$$M = -13 + 23 = \mathbf{10}$$

c) Intermediate Steps (Extended Euclidean Algorithm)

To find the inverse of $16 \pmod{23}$, we solve $16d \equiv 1 \pmod{23}$.

1. Euclidean Algorithm:

$23 = 1 \cdot 16 + 7$
$16 = 2 \cdot 7 + 2$
$7 = 3 \cdot 2 + \mathbf{1}$

2. Back-Substitution:

1 = 7 - 3(2)
1 = 7 - 3(16 - 2 ·7)
$1 = 7 \cdot 7 - 3 \cdot 16$
$1 = 7(23 - 16) - 3 \cdot 16$
$1 = 7 \cdot 23 - 10 \cdot 16$

The coefficient of 16 is -10. Thus:

$$16^{-1} \equiv -10 \equiv 13 \pmod{23}$$